The Faculty of Science and Engineering

Department of Electrical and Electronic Engineering



EEEE4077 Mechatronics Project
Project Outline and Planning Report

Vision-Augmented Mobile Robot Navigation in Dynamic Environments

AUTHOR	:	Nicholas Khor Han Le
ID	:	20617556
SUPERVISOR:	Dr. Shalini Darmaraju	
MODERATOR	:	Ir Dr. Daniel Pu
DATE	:	28 October 2026

Statement of Originality

University of Nottingham | Faculty of Science and Engineering | Department of Electrical and Electronic Engineering | EEEE4077 Mechatronics Project

- I have read and understood the University of Nottingham Academic Misconduct Policy.
- I am aware that failure to act in accordance with the Regulations Governing Academic Integrity may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties to verify whether my work contains plagiarised material, and for quality assurance purposes.

Note: *You must change the statements in the boxes to reflect your individual submission.*

1. Acknowledgment of Sources

You must acknowledge all sources of information (e.g. ideas, algorithms, data) using citations to auditable, authoritative sources. You must also put quotation marks around any sections of text that you have copied without paraphrasing and cite the original source. If any figures or tables have been taken or modified from another source, you must explain this in the caption, and cite the original source.

Student Declaration: I have acknowledged all sources, and identified any content taken from elsewhere. All non-original ideas, algorithms, mathematical formulations, and published benchmarks are formally cited to auditable, authoritative academic literature using IEEE-style numerical citations ([1]–[13]). No text has been copied verbatim without quotation and attribution. All system architecture and workflow diagrams (Figures 1 and 2) and schedule tables were authored directly by me for this project.

2. Resources Produced by Someone Else

If you have used any code (e.g., open-source code), reference designs, or similar resources that have been produced by anyone else, you must list them in the box below. In the report, you must explain what was used and how it relates to the work you have done.

Student Declaration: I have used the following resources that have been produced by someone else:

1. Robot Operating System 2 (ROS 2 Jazzy Jalisco / Humble): Open-source robotics middleware providing message passing, lifecycle nodes, and coordinate transform (tf2) management.

2. Nav2 Navigation Stack (nav2_mppi_controller, nav2_costmap_2d, nav2_bringup): Standard open-source mobile robot navigation architecture. The native MPPI trajectory rollout infrastructure and 2D costmap servers are leveraged directly.

3. Gazebo Harmonic: Open-source physics simulator utilized for world loading, differential-drive mobile robot dynamics, laser/camera sensor simulation, and dynamic pedestrian actor execution.

4. Ultralytics YOLOv8 (yolov8n pre-trained weights): Open-source neural network weights and architecture used strictly as an off-the-shelf 2D visual detector for bounding box and semantic class generation.

5. ByteTrack / BoxMOT: Open-source 2D multi-object tracking implementation used for frame-to-frame bounding box data association in image pixel space.

6. Open-Source Scientific & Math Libraries: OpenCV (image handling and camera projection geometry), Eigen3 (C++ linear algebra matrix calculations), NumPy, and SciPy (data evaluation and statistical plotting).

I have acknowledged all input (hardware, software, intellectual) taken from elsewhere and explained how this was used. All custom sensor fusion logic (camera–LiDAR frustum gating), linear Kalman filter velocity estimators, custom C++ dynamic obstacle MPPI critic plugin (*DynamicObstaclesCriticPlugin*), and automated evaluation test harnesses are developed by me.

3. Generative AI Declaration

If you have used generative AI tools in the preparation of your assessed work, or the underlying code/designs/underpinning work, its use must be declared in a signed statement in the box below. You must declare the tools employed and the method and extent of use. You can do this using the table below:

Generative AI tools have been used in the preparation of this work as follows:

Tool	Application Scope	Nature of Assistance & Verification
Google Gemini	Drafting boilerplate configuration and editorial structuring	Used to generate standard ROS 2 node boilerplate, CMakeLists.txt templates, and package.xml dependency structures. Used iteratively to improve grammatical conciseness, structural cohesion, and technical phrasing across draft report sections. All generated templates and text were critically reviewed, manually edited, debugged, and integrated by the author. System architecture and design choices remain solely the author's work.
Microsoft Copilot / Anthropic Claude	Syntax reference and proofreading	Used for target syntax lookups (e.g., C++17 idiom verification, ROS 2 parameter declaration schemas) and line-level proofreading of draft prose.

Student Declaration: *"I confirm that all sources and forms of assistance have been fully acknowledged. Generative AI tools were employed strictly in an assistive capacity for syntax reference, boilerplate generation, and editorial refinement. All system engineering, conceptual design, mathematical derivations, analysis, and final conclusions represent my own original intellectual work, and I accept full responsibility for the technical accuracy and integrity of all submitted content."*

4. Independent Work & External Assistance

You can consult with teaching staff/demonstrators, but you should not show anyone else your work (this includes uploading your work to publicly accessible repositories e.g. GitHub, unless expressly permitted by your supervisor). Nor should you help other students with their work. You must get permission in writing before you seek outside assistance, e.g. a proofreading service, and declare it here.

Student Declaration: I did all the work myself, and have not helped anyone else. No external proofreading services, essay mills, or paid third-party contractors were engaged. The code repository is maintained privately and has not been distributed or made publicly accessible without explicit supervisory permission.

5. Research Data Integrity & Reproducibility

You must not have fabricated, modified or distorted any data, evidence, references, experimental results, or other material used or presented in the report. You must clearly describe your experiments and how the results were obtained, and include all data, source code and/or designs (submitted as a separate file) so that your results could be reproduced.

Student Declaration: The material in the report is genuine, and I have included all my data/code/designs. All simulated sensor parameters, mathematical formulations, and evaluation metrics are truthful and reproducible within the containerized simulation environment.

6. Ethical Approval

If your work involved research studies (including surveys) on human participants, their cells or data, or on animals, you must have been granted ethical approval before the work was carried out, and any experiments must have followed these requirements. You must give details of this in the report, and list the ethical approval reference number(s) in the box below.

Student Declaration: **Not applicable.** This project is conducted entirely within computer simulation (Gazebo Harmonic and ROS 2) utilizing synthetic actor models. It does not involve human subjects, live animal testing, clinical trials, or personal identifiable data.

Supervisory Discussion & Final Sign-Off

My responses to these questions are full and complete, and I have discussed them with my supervisor.

Field	Detail
Student Name	Nicholas Khor Han Le
Student ID	20617556
Degree Programme	MEng (Hons) Electrical and Electronic Engineering / Mechatronics
Project Code & Title	SD-MEng-26-01: Vision-Augmented Mobile Robot Navigation in Dynamic Environments
Supervisor	Dr. Shalini Darmaraju
Moderator	Ir Dr. Daniel Pu
Date	2 November 2026
Student Signature	*Nic*

Table of Contents

Contents

Statement of Originality....2
1. Overview of the Project....5
1.1 Commercial Context and the Freezing Robot Bottleneck....5
1.2 Project Aim and Core Objectives....5
1.3 System Architecture, Component Requirements, and Dependencies....6
2. Specification....7
3. Methodology....8
3.1 Development Lifecycle, Technical Stack, and Skill Assessment....8
3.2 Workflow Interdependencies and Algorithmic Trade-Offs....9
4. Originality....10
4.1 Novelty in System Architecture and Practical Application Focus....10
4.2 Acknowledged Project Limitations and Quantified Impact on Specification....11
5. Risk Management....11
6. Planned Work Schedule....12
References....14
Appendix A: Supervisor-Provided Project Specification....15
A.1 Project Description....15
A.2 Target Sustainable Development Goals (SDGs)....15
A.3 Project Objectives....15
A.4 Project Deliverables & Resource Breakdown....15

1. Overview of the Project

1.1 Commercial Context and the Freezing Robot Bottleneck

According to the International Federation of Robotics (IFR), transportation and logistics represented the largest commercial application class for professional service robots in 2024, recording 102,900 unit deployments globally—a 14% year-on-year expansion [1]. The vast majority of these Autonomous Mobile Robots (AMRs) operate in *brownfield* industrial facilities: pre-existing warehouses, distribution hubs, and manufacturing shop floors that cannot be rebuilt around segregated automated lanes. Instead, AMRs must navigate narrow corridors shared with dynamic human workers, pallet jacks, and forklifts. To maintain commercial cost viability and adhere to industrial functional safety standards (ISO 3691-4 / SIL-2), brownfield AMRs predominantly pair a single-plane (2D) safety laser scanner with the open-source Robot Operating System 2 (ROS 2) Navigation stack, Nav2 [2], [3]. While planar 2D LiDAR provides millimetre-accurate Time-of-Flight (ToF) ranging over wide horizontal fields of view (270°–360° at 20–40 Hz), coupling planar range sensing with standard 2D occupancy grids (nav2_costmap_2d) introduces a critical operational bottleneck in dynamic environments: the **Freezing Robot Problem** [4].

Formally identified by Trautman and Krause [4], the *freezing robot problem* occurs when an autonomous mobile robot navigating amidst moving traffic halts, oscillates, or executes erratic recovery spins because its planner evaluates all forward trajectories as unsafe:

- **Planar Point Fragmentation and Semantic Blindness:** A 2D LiDAR mounted 10–20 cm above the floor intersects a walking pedestrian as two sparse, oscillating point clusters (lower-limb cross-sections) lacking semantic class identity, bounding volume, or velocity vectors.
- **Static Costmap Representation:** Standard navigation pipelines insert range scans directly into a 2D occupancy grid [2], treating every detected cell as an immovable static barrier. When a pedestrian crosses a corridor ahead at normal walking speed (1.2–1.4 m/s [9]), the planner cannot anticipate that the obstacle will clear the path within 1–2 seconds, causing forward candidate trajectories to halt.

1.2 Project Aim and Core Objectives

Project Aim: *To design, implement, and evaluate in simulation a vision-augmented mobile robot navigation framework that combines camera visual information with planar range sensor measurements to track moving obstacles, estimate their positions and velocities, and incorporate short-term motion predictions into local path planning and collision avoidance.*

Core Objectives:

(O1) Multi-Modal Obstacle Detection and Tracking: To develop an indoor mobile robot simulation and adapt visual detection and multi-object tracking methods to combine simulated camera images with planar range measurements for obstacle position and velocity estimation.

(O2) Motion-Aware Local Path Planning and Collision Avoidance: To develop a local navigation controller that incorporates short-term obstacle motion predictions and expanding safety margins into trajectory evaluation to eliminate robot freezing while maintaining collision-free navigation.

(O3) Systematic Comparative Evaluation: To evaluate the proposed framework against range-sensor-only navigation and non-predictive visual detection baselines across varying

obstacle speeds, crossing patterns, and densities, assessing tracking accuracy, navigation efficiency, and computation time.

1.3 System Architecture, Component Requirements, and Dependencies

Figure 1 and Table 1 summarize the top-down system architecture, decoupling static walls (handled in the 2D costmap) from moving obstacles (tracked in O1 and forward-projected in O2).

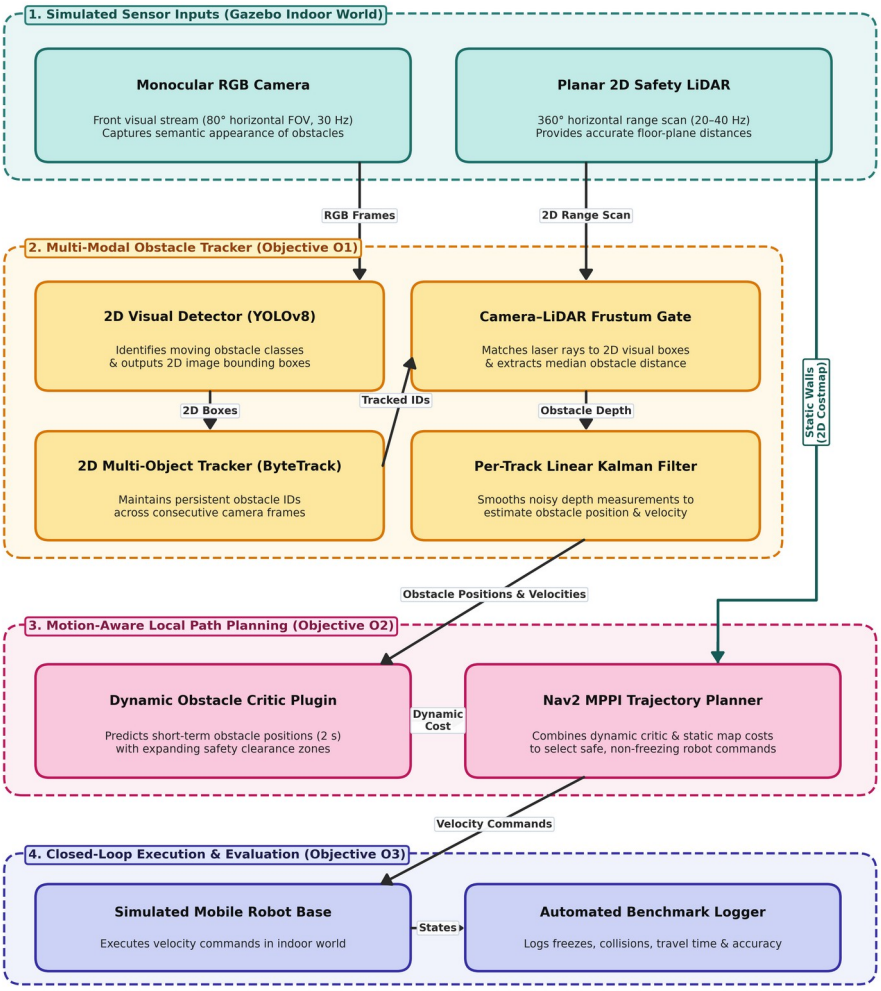


Figure 1. *Top-down system architecture block diagram showing sensor inputs, multi-modal tracking (O1), predictive local planning (O2), and evaluation logging (O3).*

Table 1. *Subsystem component requirements, interfaces, operating rates, and practical limitations.*

Subsystem Component	Input → Output Interface	Operating Rate	Key Requirement & Practical Limitation
2D Visual Detector (Simulated Camera)	Camera RGB images (1280×720) → 2D obstacle bounding boxes & class labels	≥ 30 Hz (12–18 ms)	• **Requirement:** Must run faster than the range scanner. • **Limitation:** Restricted to trained indoor obstacle classes.
2D Multi-Object Tracker (Image-Space Association)	2D bounding boxes → Persistent obstacle track IDs across frames	≥ 30 Hz (1–2 ms CPU)	• **Requirement:** Two-stage box matching preserves IDs during partial occlusion. • **Limitation:** Tracks expire if fully occluded for > 1.0 s.
Camera–LiDAR Frustum Gate (Sensor Fusion [6])	Active 2D boxes + 2D laser scan → Matched obstacle floor position	20–40 Hz (1–2 ms CPU)	• **Requirement:** Median depth filtering rejects background wall hits. • **Limitation:** Velocity tracking limited to overlapping 80° camera FOV.

Subsystem Component	Input → Output Interface	Operating Rate	Key Requirement & Practical Limitation
Linear Kalman Filter (State Estimation [7])	Matched obstacle positions → Smoothed 2D position & velocity vectors	30 Hz (< 0.2 ms CPU)	• **Requirement:** Constant-velocity model with outlier rejection gate. • **Limitation:** Valid over short horizons (≤ 2.0 s) [9]; lags on sudden turns.
Predictive Trajectory Critic (Local Planner Plugin [5], [10])	Tracked velocities + candidate trajectories → Dynamic collision cost scores	≥ 20 Hz (< 0.3 ms CPU)	• **Requirement:** Projects obstacle motion over 2.0 s with expanding safety zones. • **Architecture:** Moving agents scored by critic; static walls stay in 2D costmap.

2. Specification

Table 2 and Table 3 define the quantitative project specifications, stretch goals, and software deliverables evaluated across five indoor simulation scenarios (corridor crossings, narrow aisles, fast carts, multi-agent intersections, and static controls).

Table 2. *Project specification metrics, minimum/target thresholds, engineering justification, and test methods.*

ID	Specification Metric	Minimum	Target	Engineering Justification	Verification Method	Ch.
O1-a	Obstacle velocity error (≤ 8 m)	≤ 0.25 m/s	≤ 0.15 m/s	≈ 10% of normal 1.4 m/s walking speed [9]; keeps 2.0 s prediction error under 0.3 m.	Compare estimated velocity against simulator ground truth across ≥ 30 runs.	**M**
O1-b	Camera–LiDAR track matching	≥ 85%	≥ 95%	Accurate depth gating [6] prevents background walls from corrupting velocity estimates [7].	Percentage of visual boxes matched to a laser cluster within 0.3 m of ground truth.	**M**
O2-a	Robot freezes per 10 crossings	≥ 50% below C0	0 freezes	Directly addresses the freezing robot problem [4] where static planners halt or spin.	Count planner timeouts, recovery spins, and prolonged stops (> 2.0 s).	**H**
O2-b	Route completion time vs. baseline	≥ 10% faster	15–25% faster	Smooth speed adjustment and passing behind moving obstacles avoid stop-start delays.	Compare mean travel time across ≥ 30 seeded trials per scenario.	**M**
O2-c	Collision-free navigation	0 collisions	0 collisions	Predictive motion planning must never compromise physical collision safety.	Monitored continuously via simulation contact sensors; any contact fails the trial.	**M**
O3-a	3-Way ablation separation	$p < 0.05$	$p < 0.01$	Distinguishes the effect of visual detection alone (C1) vs. motion prediction (C2).	Paired t-test and one-way ANOVA across C0, C1, and C2 over speeds of 0.5–2.0 m/s.	**M**
O3-b	Cumulative waiting time per run	≥ 50% reduction	≤ 5.0 s	Measures reduction in intersection deadlocks during dynamic cross-traffic.	Total duration the robot remains stationary while a navigation goal is active.	**M**
O3-c	Real-time update frequency	≥ 15 Hz	≥ 20 Hz	Planner must match the 20 Hz laser scan rate for responsive trajectory updates [10].	Mean execution rate of tracker (≥ 30 Hz) and local planner (≥ 20 Hz).	**L**
NF1	Containerised reproducibility	Pass	Pass	Ensures reproducible simulation benchmarks across workstations.	Single launch command inside a pinned Docker container reproduces all logs.	**L**

Table 3. *Prioritized stretch goals (S1–S3) and core software deliverable mapping (O1–O3).*

ID / Obj.	Stretch Goal or Core Deliverable	Trigger Condition / Software Artifact	Validation Criterion
S1 (Stretch)	Dense Crowd Stress Test (4–6 moving obstacles)	Triggered if core O1 & O2 targets are met on schedule.	Evaluate multi-object tracking stability and planner success under heavy occlusion.
S2 (Stretch)	High-Speed (2.5 m/s) & Diagonal Crossings	Triggered if standard scenarios complete ahead of schedule.	Identify performance limits of the constant-velocity motion model.
S3 (Stretch)	CPU-Only Resource Profiling (4-core limit)	Triggered during final benchmark analysis phase.	Verify ≥ 20 Hz local planning rate under constrained CPU resources.

ID / Obj.	Stretch Goal or Core Deliverable	Trigger Condition / Software Artifact	Validation Criterion
O1 Deliv.	Vision–LiDAR Obstacle Tracker Package	ROS 2 perception & Kalman tracking node with source code.	Verified against simulation ground truth across ≥ 30 trials (O1-a, O1-b).
O2 Deliv.	Predictive Local Navigation Plugin	C++ dynamic obstacle critic plugin + costmap fallback layer.	Closed-loop crossing trials verifying 0 collisions and reduced freezing (O2-a/b/c).
O3 Deliv.	Comparative Benchmark Suite & Final Report	Automated scenario runner, statistical scripts & demo videos.	Statistically significant comparison across C0, C1, and C2 + final dissertation.

3. Methodology

3.1 Development Lifecycle, Technical Stack, and Skill Assessment

To achieve Objectives O1–O3, the simulation framework is built in **Gazebo Harmonic** and **ROS 2 Jazzy**. Objective O1 pairs **YOLOv8n** visual detection and **ByteTrack** [8] 2D multi-object association with **Zhang–Pless** [6] camera–LiDAR frustum depth gating and a **Constant-Velocity Linear Kalman Filter** [7]. Objective O2 implements a native C++17 **DynamicObstaclesCriticPlugin** inside **nav2_mppi_controller** [5], [10], and Objective O3 benchmarks **C0 (2D LiDAR-only baseline)**, **C1 (Visual detection at t = 0 without motion prediction)**, and **C2 (Full predictive MPPI navigation)** across steps M0–M5 (Table 4).

Table 4. *Methodology steps (M0–M5), selected software tools, practical limitations, and student skill assessment.*

Step	Core Process & Selected Tools	Tool Limitations & Project Impact	Student Skill Level & Preparation Tasks
M0: Setup	Ubuntu 24.04 LTS, ROS 2 Jazzy, Gazebo Harmonic, Docker container.	Older ROS 2 distributions reach end-of-life by 2027; containerisation prevents driver drift.	**Challenge: Low.** Existing Linux, Docker, and ROS 2 workspace fluency.
M1: Sim & C0	Gazebo indoor corridor worlds, differential-drive robot model, moving obstacle actors, Nav2 baseline.	Scripted actors follow fixed paths unless randomised; contact sensors must be explicitly configured.	**Challenge: Medium.** Build modular actor spawners with seeded speeds (0.5–2.0 m/s) and log C0 baseline.
M2: Tracker (O1)	YOLOv8n detector, ByteTrack [8] 2D tracker, Zhang–Pless frustum projection [6], Linear Kalman Filter [7].	Camera FOV (80°) is narrower than LiDAR (360°); laser rays can bleed past narrow legs onto walls.	**Challenge: Low.** Strong computer vision & statistics background; add median depth filtering and outlier gating.
M3: Critic (O2)	Native C++17 DynamicObstaclesCriticPlugin inside Nav2 MPPI controller [5], [10].	Evaluating 2,000 candidate trajectories per cycle must finish well within the 50 ms (20 Hz) budget.	**Challenge: High (first Nav2 C++ plugin).** Prototype a minimal C++ critic plugin early to verify pluginlib integration.
M4: Harness (O3)	Automated Python scenario runner, Gazebo reset services, rosbag2 logger, ground-truth evaluator.	Large batch runs (450 trials across C0, C1, C2) can stall if simulator nodes fail to reset cleanly.	**Challenge: Medium.** Add process watchdog timeouts, seeded trial scripts, and headless simulation mode.
M5: Analysis (O3)	Python statistical & plotting suite (NumPy, SciPy, Matplotlib).	Small sample sizes yield noisy comparisons under multi-agent interactions.	**Challenge: Low.** Automate paired t-tests, ANOVA, and confidence interval plots from recorded logs.

3.2 Workflow Interdependencies and Algorithmic Trade-Offs

Figure 2 illustrates the top-down task dependency flow (including the parallel benchmark harness M4 and costmap fallback F1), and Table 5 summarizes the algorithmic trade-offs across perception, prediction, and control.

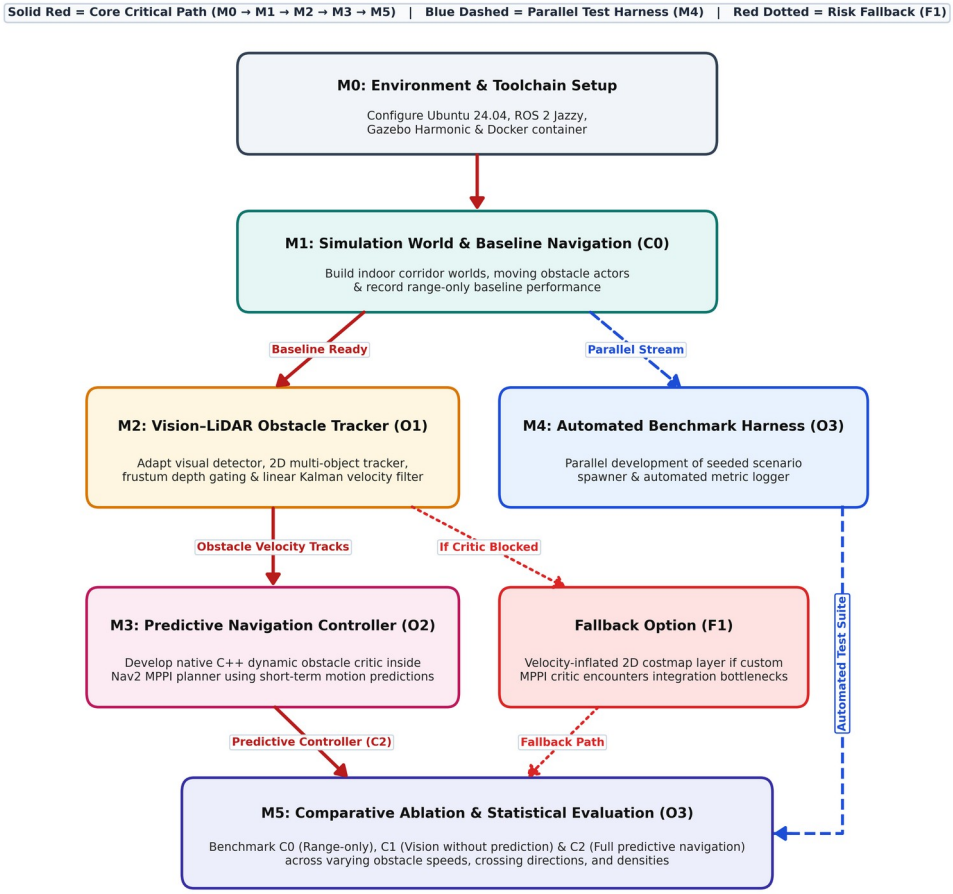


Figure 2. *Top-down methodology workflow showing the core critical path (solid red), parallel test harness (dashed blue), and fallback path (dotted red).*

Table 5. *Algorithmic selection trade-offs across obstacle tracking, motion prediction, and local trajectory planning.*

Subsystem	Selected Method	Evaluated Alternatives	Practical Engineering Justification
1. Detection & Depth Fusion (O1)	**YOLOv8n + 2D ByteTrack [8] + Camera–LiDAR Frustum Gate [6]** (< 3 ms CPU association)	• 2D Laser Clustering only • DeepSORT (CNN ReID) • 3D Point Cloud Networks	2D laser-only clustering splits walking legs and lacks semantic class identity. 3D neural networks require heavy GPU memory. 2D ByteTrack maintains stable IDs on CPU, while frustum median gating extracts accurate laser depth.
2. Obstacle Motion Model (O1/O2)	**Constant-Velocity (CV) Linear Kalman Filter [7], [9]** (< 0.2 ms CPU)	• Social Force Model [9] • Interactive Gaussian Processes [4] • Deep Trajectory Transformers	Indoor pedestrians and carts maintain steady velocity over short 1–2 s windows [9]. Evaluating complex interactive or neural models across thousands of trajectory rollouts exceeds the 20 Hz budget, whereas linear extrapolation is instantaneous.
3. Local Trajectory Planner (O2)	**Nav2 MPPI Planner + Custom Dynamic Critic [5], [10]** (< 15 ms total CPU loop)	• Dynamic Window (DWA) • Timed Elastic Band (TEB) • Post-Filter Barrier Functions [11]	DWA samples constant-velocity arcs and freezes at crossings. TEB can oscillate in multi-agent intersections. Scoring predicted obstacle positions directly inside MPPI smoothly guides trajectories behind moving obstacles.

4. Originality

4.1 Novelty in System Architecture and Practical Application Focus

Table 6 contrasts this project against standard industrial baselines and recent high-compute GPU-dependent navigation methods that are unoptimised for practical onboard deployment.

Table 6. *Comparison against standard industrial navigation, non-predictive trackers, and high-compute GPU MPPI variants.*

Dimension	Standard Industrial AMR (Nav2 + 2D LiDAR [2])	Vision–LiDAR Trackers (Without Prediction)	High-Compute / GPU MPPI Variants [11]–[13]	This Project (Proposed Framework)
Sensor Hardware	Planar 2D LiDAR only (≈ $400)	2D/3D LiDAR + RGB/Depth ($500–$4,000)	Dense 3D LiDAR / Depth Point Clouds (> $3,000)	**2D LiDAR + Monocular Camera Retrofit (≈ $450)**
Obstacle Representation	Static 2D costmap cells fixed at current instant	Tracked boxes written into static 2D costmap	Unsegmented 3D point clouds [12] or reactive barriers [11]	**Decoupled static costmap + moving velocity tracks**
Trajectory Scoring	Static costmap critics only → freezes in traffic	Static clearance inflation + raycast clearing lag	Heavy GPU point-cloud SDF [12] or GPU clustering [13]	**Lightweight C++ MPPI Critic with 2 s motion prediction**
Compute Footprint	Standard onboard CPU	CPU + Edge GPU	High-compute dedicated CUDA GPU (unoptimised for edge)	**Standard CPU for control (< 0.3 ms) + light detector**
Evaluation Design	Static obstacle tests	Isolated perception tracking metrics only	Monolithic comparison against legacy planners	**Controlled 3-way ablation (C0 vs. C1 vs. C2)**

- **Practical Open-Source Controller Extension:** All 11 default critics in Nav2 MPPI score trajectories only against static 2D costmaps [10]; adding a modular C++ dynamic obstacle critic resolves robot freezing without altering existing map pipelines.
- **Low-Cost Sensor Complementarity:** Pairing a standard 2D safety laser scanner with a single monocular camera provides semantic class filtering and accurate range-gated velocity tracking without expensive 3D LiDAR hardware [1].
- **Application-Optimised vs. High-Compute GPU Approaches:** Recent MPPI extensions such as SE-MPPI [11], EXACT-MPPI [12], and CSC-MPPI [13] rely on heavy GPU compute (CUDA point-cloud signed distance fields or GPU clustering) that is unoptimised for practical mobile robot deployment, whereas our velocity-projected critic runs in < 0.3 ms on CPU.
- **Controlled 3-Way Experimental Ablation:** Comparing C0 (range-only), C1 (visual detection without motion prediction), and C2 (full predictive planning) directly isolates the contribution of motion prediction.

4.2 Acknowledged Project Limitations and Quantified Impact on Specification

- **Simulation-Only Validation (0% Hardware):** Simulated actors lack subtle human gait variations and optical glare; accordingly, Table 2 defines both a conservative Minimum pass threshold (≥ 85% matching) and a Target goal (≥ 95%).
- **Planar Laser Height & 80° Camera FOV:** Velocity tracking is active within the overlapping 80° front camera FOV; outside this cone, the robot safely defaults to the 360° 2D LiDAR static costmap to guarantee zero collisions.
- **Short-Term Constant-Velocity Horizon (≤ 2.0 s):** Linear velocity predictions diverge if an obstacle abruptly reverses direction; bounding the prediction horizon to 2.0 s with expanding safety margins prevents over-confident swerving.

5. Risk Management

Table 7 summarizes the four core technical and project risks, rated by Likelihood (L, 1–5), Impact (I, 1–5), and composite Score (L × I), alongside practical mitigation and fallback strategies.

Table 7. *Core technical and project risk register with likelihood, impact, system effect, and mitigation strategies.*

ID	Risk Description	L	I	Score	Impact on System	Mitigation & Fallback Strategy
R1	Predictive C++ MPPI critic plugin suffers integration faults or exceeds real-time CPU budget (O2).	4	5	**20**	Controller loop drops below 20 Hz target or fails to load in Nav2.	Vectorise trajectory cost calculations over a bounded 2.0 s horizon. **Fallback (F1):** Inject velocity-stretched safety zones into the standard 2D costmap.
R2	Camera–LiDAR projection error or laser rays bleeding onto background walls corrupt velocity tracks (O1).	3	4	**12**	Noisy obstacle velocity estimates degrade prediction accuracy.	Use exact simulation sensor transforms, apply median depth filtering inside each 2D bounding box frustum, and reject outlier jumps in the Kalman filter.
R3	Simulation environment instability or scope expansion across multi-scenario batch testing (O3).	3	4	**12**	Incomplete comparative ablation runs or environment crashes.	Lock all ROS 2 and Gazebo dependencies in a reproducible Docker container, automate trial watchdog resets, and gate stretch scenarios behind core C0–C2 completion.
R4	Visual occlusion between multiple crossing obstacles causes track ID switching (O1/O2).	3	3	**9**	Temporary loss of velocity vector during dense multi-agent crossings.	Use two-stage ByteTrack association [8] to retain low-confidence occluded boxes, paired with short-term Kalman state coasting (up to 1.0 s).

6. Planned Work Schedule

Table 8 (Gantt chart) and Table 9 outline the 30-week work schedule (450 hours total at 15 hours/week) across the Autumn Semester (Weeks 1–15) and Spring Semester (Weeks 16–30).

Table 8. *Project Gantt chart across the Autumn Semester (Weeks 1–15) and Spring Semester (Weeks 16–30).*

Project Task	AUTUMN SEMESTER (Weeks 1–15)							SPRING SEMESTER (Weeks 16–30)						
General Activity	W1-2	W3-4	W5-6	W7-8	W9-10	W11-12	W13-15	W16-18	W19-20	W21-22	W23-24	W25-26	W27-28	W29-30
1. Setup environment & Docker (W1–W3)														
2. Simulate world, actors & baseline (W3–W6)														
3. Prepare obstacle tracker (vision + LiDAR) (W7–W12)														
4. Setup automated testing harness (W8–W18)														
5. Develop predictive navigation controller (W13–W18)														
6. Run comparative simulation benchmarks (W19–W24)														
7. Analyze results & statistical metrics (W22–W26)														
8. Write final thesis & prepare video demos (W25–W30)														

Table 9. *Summary of planned project tasks, semester timeline, allocated hours, and key milestones.*

Semester	Weeks	General Task Description	Hours	Key Deliverable & Milestone
Autumn Semester	W1–W3	**Setup environment & Docker**	45 h	Ubuntu 24.04, ROS 2 Jazzy, Gazebo Harmonic & reproducible Docker container ready.
Autumn Semester	W3–W6	**Simulate world, actors & baseline**	45 h	Indoor corridor world, moving obstacle actors, and C0 range-only baseline logged.

Semester	Weeks	General Task Description	Hours	Key Deliverable & Milestone
Autumn Semester	W7–W12	**Prepare obstacle tracker (vision + LiDAR)**	90 h	Multi-modal tracker (O1) publishing smoothed obstacle positions and velocities.
Autumn / Spring	W8–W18	**Setup automated testing harness**	45 h	Parallel scenario spawner, automated trial runner, and metric logger ready.
Autumn / Spring	W13–W18	**Develop predictive navigation controller**	75 h	Closed-loop predictive MPPI critic (O2) running at ≥ 20 Hz alongside C1 baseline.
Spring Semester	W19–W24	**Run comparative simulation benchmarks**	60 h	Batch simulation trials completed across C0, C1, and C2 under varying speeds/densities.
Spring Semester	W22–W26	**Analyze results & statistical metrics**	30 h	Statistical comparisons (t-test / ANOVA), performance plots, and stretch profiling.
Spring Semester	W25–W30	**Write final thesis & prepare video demos**	60 h	Final MEng dissertation report, documented source code, and simulation video demos.

References

[1] International Federation of Robotics (IFR), "World Robotics 2025 – Service Robots," Frankfurt, Germany, Press Release, 2025. [Online]. Available: https://ifr.org/ifr-press-releases/news/service-robots-see-global-growth-boom

[2] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The Marathon 2: A Navigation System," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, Las Vegas, NV, USA, 2020, pp. 2718–2725, arXiv:2003.00368.

[3] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "From the Desks of ROS Maintainers: A Survey of Modern & Capable Mobile Robotics Algorithms in the Robot Operating System 2," *Robotics and Autonomous Systems*, vol. 168, p. 104496, 2023, arXiv:2307.15236.

[4] P. Trautman and A. Krause, "Unfreezing the robot: Navigation in dense, interacting crowds," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, Taipei, Taiwan, 2010, pp. 797–803, doi:10.1109/IROS.2010.5654369.

[5] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, "Information theoretic MPC for model-based reinforcement learning," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, Singapore, 2017, pp. 1714–1721, doi:10.1109/ICRA.2017.7989202.

[6] Q. Zhang and R. Pless, "Extrinsic calibration of a camera and laser range finder (improves camera calibration)," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, Sendai, Japan, 2004, pp. 2301–2306, doi:10.1109/IROS.2004.1389752.

[7] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.

[8] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in *Proc. European Conf. Computer Vision (ECCV)*, Tel Aviv, Israel, 2022, pp. 1–21, arXiv:2110.06864.

[9] D. Helbing and P. Molnár, "Social force model for pedestrian dynamics," *Physical Review E*, vol. 51, no. 5, pp. 4282–4286, 1995, doi:10.1103/PhysRevE.51.4282.

[10] S. Macenski and Nav2 Development Team, "Nav2 MPPI Controller: Model Predictive Path Integral Trajectory Optimization," ROS 2 Navigation Stack Documentation, 2024. [Online]. Available: https://github.com/ros-navigation/navigation2/tree/main/nav2_mppi_controller

[11] J. M. Kang, "SE-MPPI: Safe-Escape Nav2-Native MPPI Controller with Control Barrier Functions," GitHub Repository / Technical Preprint, 2024. [Online]. Available: https://github.com/kjungmo/se-mppi-nav2

[12] C. Peng et al., "EXACT-MPPI: Exact Signed-Distance Navigation for Arbitrary-Footprint Robots from Point Clouds via Path Integral Control," Technical Preprint / GitHub Repository, 2024. [Online]. Available: https://github.com/caseypen/EXACT-mppi

[13] Robot Control and Intelligence Lab (RCILab), "Constrained Sampling MPPI Framework with DBSCAN for Reliable Obstacle Avoidance," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2025.

Appendix A: Supervisor-Provided Project Specification

Project Title: Vision-Augmented Mobile Robot Navigation in Dynamic Environments
Project Code: SD-MEng-26-01 | **Supervisor:** Dr Shalini Darmaraju | **Moderator:** Ir Dr Daniel Pu | **Date:** 19 October 2026

A.1 Project Description

Autonomous mobile robots need to navigate safely and efficiently in environments containing moving obstacles. Planar range sensors provide distance measurements, while cameras provide visual information that can help identify relevant obstacles. Combining these inputs with obstacle motion estimation can support navigation decisions when obstacles cross or approach the robot's planned route.

This project aims to develop a simulation-based navigation framework that combines camera information with planar range measurements for obstacle tracking and motion-aware navigation. A mobile robot equipped with simulated sensors will operate in a known indoor environment containing selected types of moving obstacles. Existing visual detection and tracking methods will be adapted and combined with range measurements to estimate obstacle positions and velocities. A simple motion model will predict their short-term positions, which will be incorporated into local path planning and collision avoidance using an existing navigation framework.

The proposed system will be evaluated against range-sensor-based navigation, with additional comparisons to distinguish the effects of visual information and motion prediction. Experiments will vary obstacle speeds, movement patterns and density. Evaluation will assess tracking accuracy, navigation success, collisions, travel time, waiting time and computational efficiency. All development and evaluation will be conducted in simulation, with images and range measurements generated by simulated sensors.

A.2 Target Sustainable Development Goals (SDGs)

- **SDG 9:** Industry, Innovation and Infrastructure
- **SDG 8:** Decent Work and Economic Growth

A.3 Project Objectives

- 1. To review existing methods for vision-augmented mobile robot navigation, camera and range-sensor integration, obstacle tracking and motion prediction.
- 2. To develop an indoor robot simulation and adapt existing detection and tracking methods to combine simulated camera and planar range measurements for obstacle position and velocity estimation.
- 3. To develop a navigation framework that incorporates short-term obstacle motion predictions into local path planning and collision avoidance.
- 4. To evaluate the proposed framework against range-sensor-based navigation under varying obstacle movements and densities, assessing tracking accuracy, navigation performance and computational efficiency.

A.4 Project Deliverables & Resource Breakdown

- 1. A literature review of vision-augmented mobile robot navigation, camera and range-sensor integration, obstacle tracking and motion prediction methods.
- 2. A configurable indoor robot simulation and an implemented obstacle detection and tracking system using simulated camera and range measurements, with documented source code.
- 3. A navigation prototype integrating short-term obstacle motion prediction with local path planning and collision avoidance.
- 4. Comparative experimental results and analysis of tracking accuracy, navigation success, collisions, travel time, waiting time and computation time, supported by reproducible simulation scenarios and video demonstrations.
- 5. A final project report documenting the literature review, methodology, system development, evaluation results and conclusions.

Resource Ratio: Hardware 0% | Software 60% | Research/Investigation/Design 40%. **Facilities:** Computer workstation only (Ubuntu Linux, ROS 2, Nav2, Gazebo, Python/C++, NumPy, Matplotlib).